

Project ATLAS

Technical Specification & Architecture Document

Autonomous Topological Learning & Addressing System *Geometry-native infrastructure for cross-organization AI reasoning without data movement.*

Confidential — shared directly for technical evaluation. Please do not redistribute publicly.

Abstract

ATLAS is an infrastructure protocol that lets AI agents reason over enterprise data as geometric structure rather than flat text, and lets multiple organizations jointly reason over their combined data without any raw data crossing a network boundary. It is composed of three independent but interoperating systems: a topological indexing and retrieval engine (Manifold-Transformers), a privacy-preserving cross-organization matching protocol (TIP), and a sandboxed remote execution model (Sovereign-MCP). This document specifies the design of each, the reasoning behind it, its current implementation status, and the specific engineering problems that remain open.

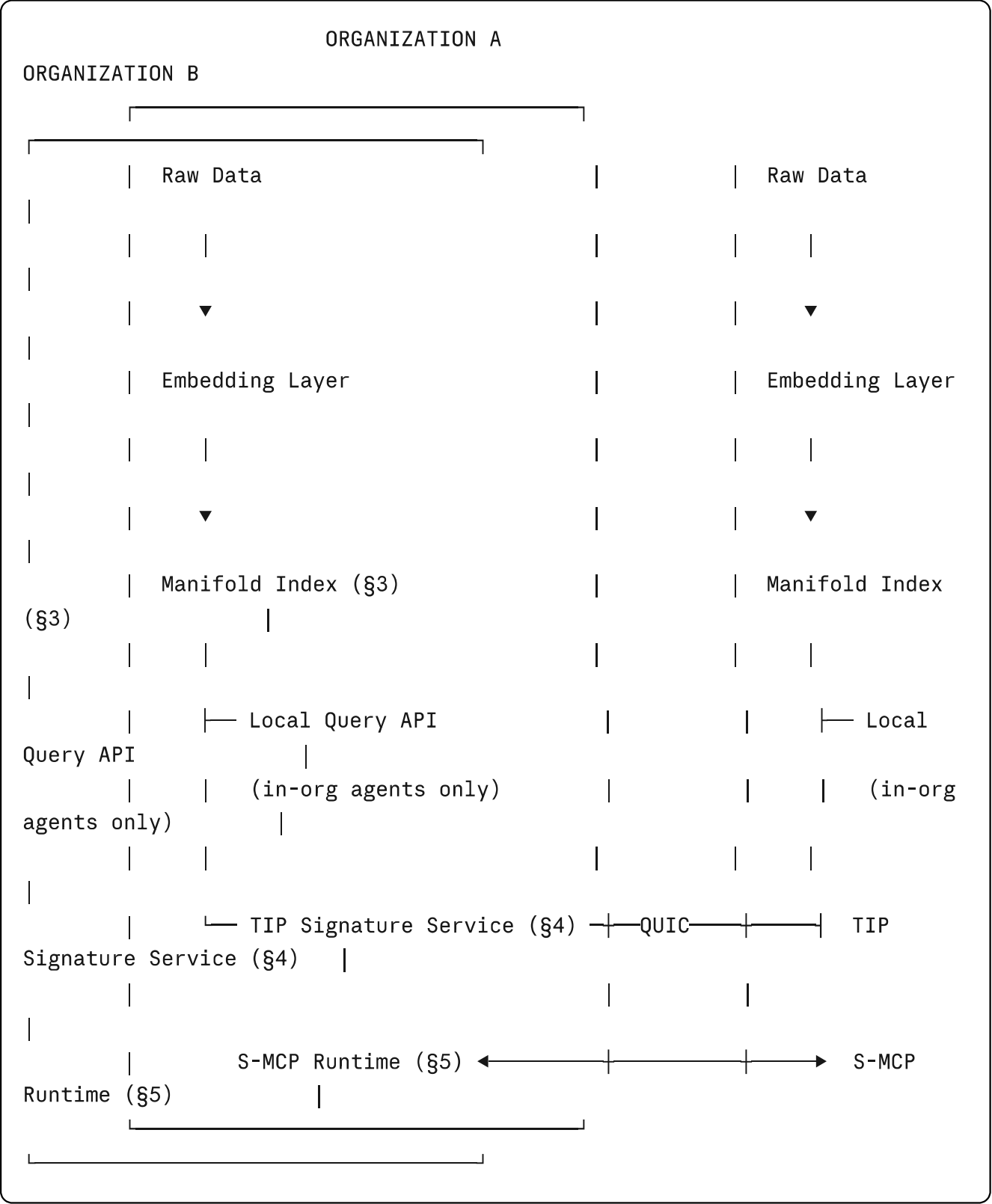
1. Motivation

Standard retrieval-augmented generation systems flatten a dataset into independent embeddings and answer queries via nearest-neighbor search under cosine similarity. This design has three structural limits that don't improve with more data or a better embedding model:

- It cannot represent relationships between documents, only similarity to a query.**
A question whose answer requires chaining facts across several records has no natural representation in a purely pointwise scoring system.
- It has no concept of structural anomaly.** Fraud, defects, and edge cases are often unusual in their relationship to their neighborhood, not necessarily in raw embedding distance from a query.
- It offers no mechanism for cross-organization reasoning without centralizing data,** which is a non-starter for regulated industries (finance, healthcare) where data residency and liability rules prevent it outright.

ATLAS is built to remove these three limits directly, rather than work around them with better prompting or bigger context windows.

2. System Architecture



Every organization runs its own node. The only things that ever leave a node are: (a) query results served to that organization's own agents, (b) hashed matching signals exchanged over TIP, and (c) small result vectors returned from remote sandboxed execution. Raw records, embeddings, and the manifold structure itself are never transmitted.

3. Component: Manifold Indexing & Retrieval

3.1 Indexing pipeline

Stage	Operation
1	Encode raw records into dense embeddings using a standard, off-the-shelf encoder
2	Build an approximate k-nearest-neighbor graph over the embedding set (HNSW)
3	Construct a Vietoris–Rips filtration over the point cloud — a sequence of simplicial complexes at increasing distance thresholds
4	Compute persistent homology across the filtration, yielding birth/death intervals for connected components, loops, and voids
5	Store the resulting complex as the primary index; retain flat embeddings only as a fallback for simple pointwise lookups

3.2 Query resolution

A query is not scored pointwise against the index. It is inserted into the existing complex, and a potential field is defined over the manifold using local point density and the persistence weight of nearby features. Retrieval proceeds as a traversal of the query along this field — conceptually similar to gradient descent — so the result is the path the query settles into, which can pass through multiple related regions in sequence rather than stopping at a single nearest match. This is the mechanism that gives multi-hop reasoning a native representation instead of requiring it to be reconstructed after the fact.

3.3 Computational cost and current status

Exact Vietoris–Rips persistence computation scales poorly — combinatorially in the number of simplices as dataset size and embedding dimension grow. This is the primary open engineering problem in the whole system. The mitigation path under active investigation:

- Sparse/witness-complex approximations to bound simplex count without materially changing the resulting topology.
- Capping homology computation at low dimensions (H_0 , H_1 , occasionally H_2), since higher-dimensional features are rarely useful for this application and are the most expensive to compute.
- Incremental (vineyard-style) updates for new data rather than full recomputation.

No claim is made yet about which of these sufficiently controls cost at production scale — that is precisely what the first implementation phase is designed to determine.

4. Component: TIP (Topology Interchange Protocol)

4.1 Design correction from earlier framing

An earlier description of this protocol characterized it as comparing global topological summaries (dataset-wide Betti numbers) between organizations. That framing does not hold up: a global invariant describes a dataset's shape as a whole and cannot localize where an overlap exists, which makes it useless for routing computation to a specific place. The specification below is the corrected mechanism, and is the one actually being implemented.

4.2 Mechanism

1. Restrict the topological computation to a **local** neighborhood around a specific query point, not the dataset as a whole.
2. Compute the local persistence diagram for that neighborhood.
3. Apply a **persistence landscape transform**, converting the diagram into a fixed-length vector that can be compared directly with another organization's vector using ordinary distance metrics.
4. Apply **locality-sensitive hashing** (random hyperplane projection) to that vector, producing a short hash code. Similar vectors are engineered to collide with high probability; the original vector cannot be recovered from the hash.
5. Exchange only hash codes between organizations over a QUIC-based transport.
6. On collision, resolve the match to a locally-scoped region identifier — meaningless outside the organization that generated it — which is then used to target remote execution (§5).

4.3 Message flow

Node A → Node B :	OFFER	{ query_ref, lsh_codes: [...] }
Node B → Node A :	MATCH	{ matched_code, region_ref }
Node A → Node B :	EXECUTE	{ signed_wasm_module, region_ref }
Node B → Node A :	RESULT	{ result_vector (≤256 bytes), attestation }

Every exchange is logged at both endpoints, so the protocol is forensically auditable after the fact — a requirement for any deployment in a regulated context.

4.4 Security properties

Property	Status	Basis
Raw data never transmitted	Held	No stage of the protocol serializes raw records or embeddings onto the wire
Original vector not recoverable from hash	Held	LSH is a one-way, lossy projection by construction
Existence of a match is revealed	Intentional, not a flaw	This is the protocol's purpose
Information leakage under repeated querying over time	Open, unresolved	Addressed in §7
Malicious remote code execution	Held	Enforced by the S-MCP sandbox (§5), not by TIP itself

5. Component: Sovereign-MCP (Remote Sandboxed Execution)

5.1 Scope correction

An earlier description suggested that "the AI compiles its own reasoning" into a portable binary and ships it to a remote host. Taken literally, that isn't feasible: frontier-scale models require far more memory and compute than a sandboxed environment can provide. What is actually specified and buildable is narrower:

A small, task-distilled inference function — not a general reasoning model — is compiled to WebAssembly and executed remotely. This function is trained ahead of time, offline, via standard knowledge distillation from a larger model, to answer one specific, pre-defined sub-question. TIP (§4) has already done the work of identifying *where* to look and *what narrow question* to ask before this component is ever invoked — the Wasm module does not need general intelligence, only accuracy on one distilled task.

5.2 Execution flow

1. The requesting organization selects or trains a distilled inference module appropriate to the sub-task.
2. The module is compiled to `(wasm32-wasi)`, quantized (int8 target), and signed.
3. The host's runtime verifies the signature and allocates a sandbox with a fixed memory ceiling, a fixed compute-step budget, and no filesystem or network capability — only a narrow, read-only handle scoped to the matched region from TIP.
4. The module executes against that scoped data view and returns a small result vector.
5. The sandbox is torn down with no persisted state.

5.3 Candidate implementation stack

Wasmtime as the sandbox runtime (WASI capability model for scoped access), with `(tract)` or `(wonnx)` as candidate paths for running a quantized ONNX model inside `(wasm32-wasi)`. None of this requires inventing new sandboxing or ML-runtime technology — it requires integrating existing, maintained tools correctly.

6. Worked Example

Two banks suspect a shared shell company laundering funds and cannot disclose transaction ledgers to each other or to a regulator directly.

1. Each bank's node maintains a manifold index of its own transaction data.
2. A regulator's agent issues a TIP query describing the anomaly pattern of interest.
3. Both banks compute local persistence landscapes around their own relevant transaction clusters and exchange only hash codes.
4. A collision is found; each bank resolves it to a specific internal region.
5. The regulator dispatches a small distilled entity-linkage module, scoped to the matched region at each bank.
6. Each bank's sandbox returns a short, anonymized result vector.
7. The regulator combines both vectors and identifies the shared entity, having received zero raw transaction records from either institution.

7. Open Engineering Problems

These are stated directly because they are unresolved, not because a stronger version of the answer is being withheld:

- **Viectoris-Rips computation cost at production scale** is unmeasured against real target dataset sizes; sparsification's tradeoffs (§3.3) are a research question, not a settled implementation detail.
- **Information leakage from repeated TIP queries over time** has no formal bound yet. The planned mitigation — rate-limiting combined with differential-privacy noise in the hash-generation step — is a reasonable, literature-backed approach, but has not been proven sufficient for this specific construction.

- **Distillation accuracy loss** for the S-MCP inference modules is unmeasured; how much of a teacher model's capability survives compression to sandbox-appropriate size is an empirical question the first implementation phase is designed to answer.
- **LSH-based matching is probabilistic.** False positives and false negatives are expected outcomes of the design, not failure states, and need to be characterized empirically rather than assumed away.

8. Proposed Repository Layout

atlas/	
├─ manifold-core/	Python/JAX – indexing, filtration, traversal
├─ tip-protocol/	Rust – landscape transform, LSH, QUIC transport
├─ smcp-runtime/	Rust + Wasmtime – sandbox, distillation
pipeline, attestation	
├─ benchmarks/	cross-cutting evaluation, external public
datasets only	
├─ docs/	architecture notes and design records
└─ examples/	simulated multi-node end-to-end demos

9. Technology Choices

Layer	Choice	Reasoning
Manifold math	Python + JAX, built on GUDHI/Ripser primitives	Reuse established, tested TDA computation rather than reimplement it from scratch
Protocol transport	Rust, tokio , quinn (QUIC)	Memory safety and async performance for a network-facing protocol handling untrusted input
Execution sandbox	Rust, Wasmtime, WASI	Mature, actively maintained sandboxing runtime with a real capability model
Distilled inference	ONNX via tract , wonnx , quantized	Established, portable path to running a small model inside a Wasm sandbox
Test orchestration	Docker / Kubernetes	Simulating realistic multi-node deployments during development

10. Implementation Roadmap

Phase 1 (Months 1–3): Implement the manifold indexing pipeline and traversal query mechanism; benchmark against standard cosine-similarity RAG on named, external multi-hop QA datasets; characterize sparsification cost/accuracy tradeoffs. This phase's benchmark result determines whether Phase 2 proceeds as scoped or the plan is revised.

Phase 2 (Months 3–5): Build the S-MCP sandbox runtime and the distillation pipeline for compact inference modules; measure accuracy retention versus the teacher model; publish a working demonstration of remote, sandboxed execution with independently verifiable zero data exfiltration.

Phase 3 (Months 5–8): Implement TIP over QUIC in full, including the landscape/LSH pipeline as a shared library; deploy three simulated nodes and run the full worked example (§6) end-to-end; commission an independent security review of the sandbox isolation boundary before any real institutional pilot.

11. Contributing

This is a pre-implementation architecture: the design above is complete, but none of the three components has a working codebase yet. People joining now are building the first version of each component, not maintaining an existing one.

Area	Primary ownership	Relevant background
Manifold-core	§3, Phase 1	Topological data analysis, persistent homology, Python/JAX
TIP protocol	§4, Phase 3	Rust, async networking, protocol design
S-MCP runtime	§5, Phase 2	Wasmtime/Wasmer, model quantization and distillation

Questions this document doesn't answer are useful signal, not a gap to apologize for — raise them directly.

12. Glossary

Term	Definition
Manifold	A geometric structure representing relationships within a dataset, used here in place of flat vector embeddings as the primary index
Vietoris-Rips complex	A simplicial complex built from pairwise distances under a threshold, computed across a filtration of increasing thresholds
Persistent homology	Tracking topological features (components, loops, voids) across a filtration to distinguish structurally significant features from noise
Persistence landscape	A transform converting a persistence diagram into a fixed-length vector suitable for direct comparison
Locality-sensitive hashing	A hashing technique where similar inputs collide with high probability while the original input cannot be recovered from the hash
TIP	The protocol specified in §4 for privacy-preserving, cross-organization structural matching
S-MCP	The remote sandboxed execution model specified in §5
Distillation	Training a small model to approximate a larger model's behavior on a narrow task

Document status: Architecture and roadmap complete. No component has a working implementation yet; Phase 1 has not started. This document will be revised as each phase produces measurable results, including results that don't support the plan as currently scoped.